

Before moving forward let us see the code for a server running on port 9000, which replies with a text 'Welcome to the World of Go Lang' when opened in a browser:

```
package main

import (
    "net/http"
)

// HttpHandler is a handler struct
type HttpHandler struct{}

// implement `ServeHTTP` method on `HttpHandler` struct
func (h HttpHandler) ServeHTTP(res http.ResponseWriter, req *http.Request) {
    // create response binary data
    data := []byte("Welcome to the world of Go Lang")
    // write `data` to response
    res.Write(data)
}
```

```
}
func main() {
    // create a new handler
    handler := HttpHandler{}
    // listen and serve on localhost
    http.ListenAndServe(":9000", handler)
}
```

As you can see above, I wrote a Go program in a file named *http-server.go*, wherein I created an *HttpHandler* and implemented the *serveHTTP* method.

When this program is run by executing the command *go run http-server.go* in the console, the main function is executed, which calls the *http ListenAndServe* method, thereby starting the server on the address provided as an argument. Inside *serveHTTP*, I created binary data, which is written to the response using the *write* method.

Upon hitting the address *http://localhost:9000* in the browser, the

data sent by the server is displayed as a response. The server continuously runs until it is forcefully stopped by typing CTRL in the server console.

The above code creates a simple server, but the problem with it is that it responds to all the requests with the same response. For example, if the URL contains */index.html* or */test.pdf*, the response is always constant.

To solve this, we need a *ServerMux*, a route based control mechanism which handles different responses for different URLs. *ServerMux* is a built-in structure present in the HTTP package, and it accepts a function for a route. When the requests match the path, the corresponding function is called to render the desired response.

For instance, */api/accounts* maps to the *accounts* function, which in turn returns *account* related information, whereas */api/subscription* maps to the *subscription* function which returns *subscription* information.

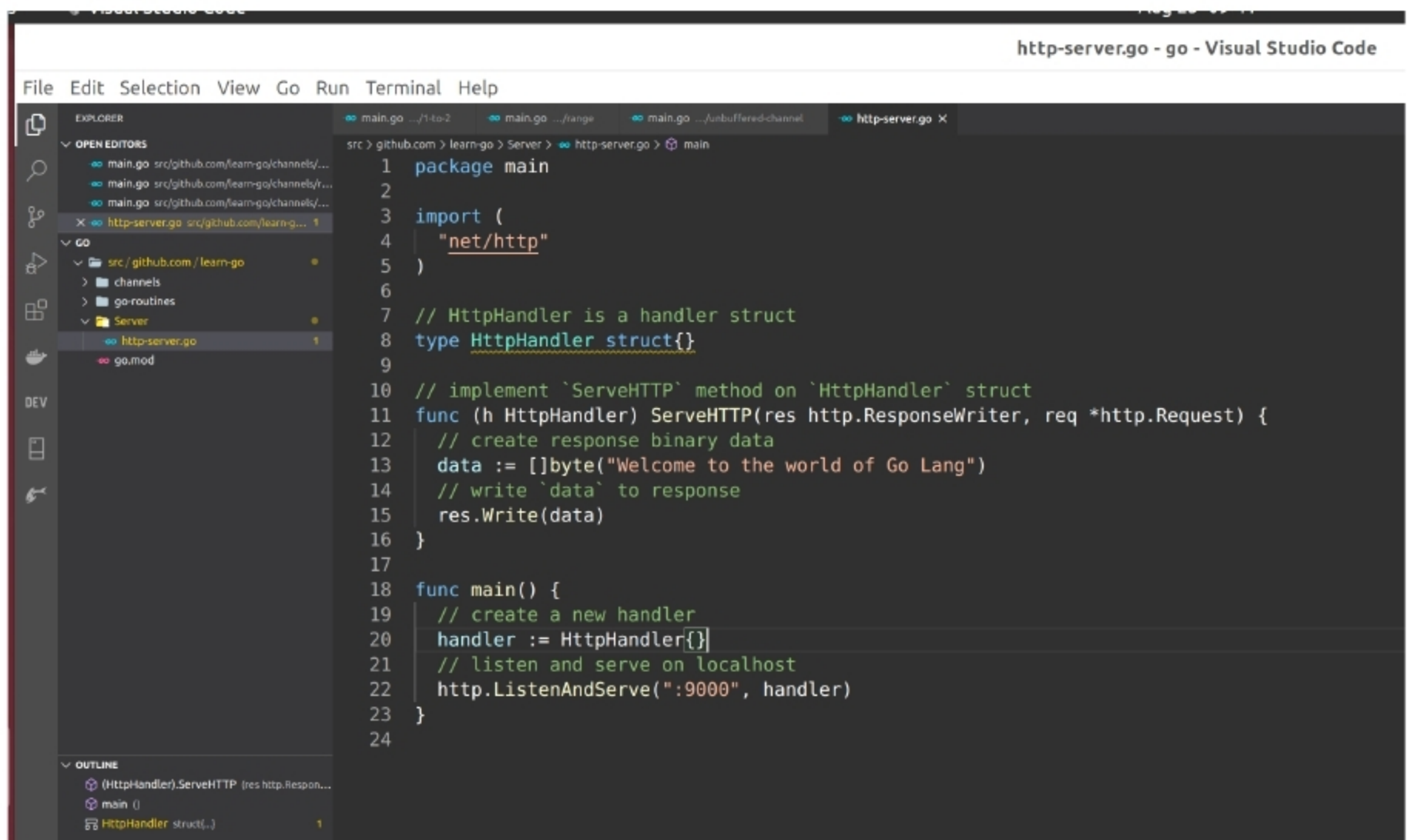


Figure 1: Code